

UNIVERSIDAD TECNOLÓGICA DE PEREIRA

Programa de Desarrollo de Software - Programación III (ProgIIIIG101)

Estudiante: Wilson Andrés Henao

Grupo: 101

Profesor: Ramiro A. Barrios Valencia

Actividad: Actividad 02 - Backtracking y SLD

1. EJERCICIO 1: PROBLEMA DE LOS PUENTES DE KÖNIGSBERG

Planteamiento: El problema consiste en determinar si es posible comenzar en una región cualquiera de Königsberg, cruzar cada uno de los 7 puentes exactamente una vez y regresar al punto de partida original.

Fundamentación Teórica e Inferencia SLD: Leonhard Euler demostró que un grafo admite un ciclo euleriano si y solo si todos sus vértices tienen un grado par. En este grafo, las 4 regiones (A, B, C y D) poseen grados impares (A=5, B=3, C=3, D=3). Al ejecutar la consulta en Prolog, el algoritmo explora iterativamente el espacio de estados usando Backtracking. Al no encontrar ningún camino válido que consuma los 7 puentes sin repetirlos, la resolución SLD concluye en fallo.

Código Prolog

```
% Hechos: Puentes y conexiones entre zonas
puente(a, b, p1). puente(a, b, p2).
puente(a, c, p3). puente(a, c, p4). puente(a, d, p5). puente(b, d, p6). puente(c, d, p7).
conecta(X, Y, P) :- puente(X, Y, P). conecta(X, Y, P) :- puente(Y, X, P). % Regla para
verificar ciclo euleriano usando backtracking
camino_euleriano(Inicio, Inicio, Usados, Total, []) :- length(Usados, Total).
camino_euleriano(Actual, Fin, Usados, Total, [P|Resto]) :- conecta(Actual, Sig, P), \+ member(P, Usados),
camino_euleriano(Sig, Fin, [P|Usados], Total, Resto).
```

Consulta y Interpretación del Resultado

```
?- camino_euleriano(a, a, [], 7, Solucion).
```

Resultado: false

Interpretación: La respuesta false obtenida en Prolog confirma formalmente la imposibilidad de resolver el problema de Königsberg, al haberse agotado el árbol de resolución SLD.

2. EJERCICIO 2: RUTAS ENTRE CIUDADES (GPS / MAPA DE ESPAÑA)

Planteamiento: Se modela el mapa de carreteras de España como un grafo no dirigido con distancias kilométricas para calcular la ruta más corta entre ciudades mediante reglas recursivas que evitan ciclos infinitos.

Código Prolog

```
% Conexiones viales y distancias (km)
tramo(coruna, vigo, 171).
tramo(coruna, valladolid, 455).
tramo(vigo, valladolid, 356).
tramo(oviedo, bilbao, 304).
tramo(bilbao, valladolid, 280).
tramo(bilbao, madrid, 395).
tramo(bilbao, zaragoza, 324).
tramo(valladolid, madrid, 193).
tramo(madrid, badajoz, 403).
tramo(madrid, jaen, 335).
tramo(madrid, albacete, 251).
tramo(madrid, zaragoza, 325).
tramo(zaragoza, barcelona, 296).
tramo(barcelona, gerona, 100).
tramo(barcelona, valencia, 349).
tramo(valencia, albacete, 191).
tramo(valencia, murcia, 241).
tramo(albacete, murcia, 150).
tramo(jaen, sevilla, 242).
tramo(jaen, granada, 99).
tramo(sevilla, cadiz, 125).
tramo(sevilla, granada, 256).
tramo(granada, murcia, 278).

distancia(X, Y, D) :- tramo(X, Y, D) ;
tramo(Y, X, D).

% Búsqueda de rutas sin ciclos con distancia total
ruta(Origen, Destino, Ruta, Distancia) :-
    obtener_ruta(Origen, Destino, [Origen], RInvertida, Distancia),
    reverse(RInvertida, Ruta),
    obtener_ruta(X, X, Vis, Vis, 0),
    obtener_ruta(Origen, Destino, Vis, Ruta, DTotal) :-
        distancia(Origen, Inter, D1),
        \+ member(Inter, Vis),
        obtener_ruta(Inter, Destino, [Inter|Vis], Ruta, D2),
        DTotal is D1 + D2.
```

Consultas y Resultados

```
% Consulta 1: Ruta óptima de Bilbao a Sevilla
?- aggregate_all(min(D, R), ruta(bilbao, sevilla, R, D), min(Dist, Ruta)).
Resultado: Dist = 857 km, Ruta = [bilbao, valladolid, madrid, jaen, sevilla].
```

Consulta	Ruta Obtenida	Distancia
Madrid a Barcelona	[madrid, zaragoza, barcelona]	621 km
Madrid a Barcelona (Alt.)	[madrid, albacete, valencia, barcelona]	791 km

3. EJERCICIO 3: GRAFO DIRIGIDO CON PESOS (CIUDADES DE CANADÁ)

Planteamiento: Se analiza la red dirigida entre las ciudades de Vancouver, Edmonton, Calgary, Saskatoon, Regina y Winnipeg, implementando reglas adicionales solicitadas.

Código Prolog con Reglas Solicitadas

```
% Arcos dirigidos con sus pesos arco(vancouver, edmonton, 16). arco(vancouver,
calgary, 13). arco(edmonton, saskatoon, 12). arco(calgary, edmonton, 4). arco(calgary,
regina, 14). arco(saskatoon, calgary, 9). arco(saskatoon, winnipeg, 20). arco(regina,
saskatoon, 7). arco(regina, winnipeg, 4). % Regla C: Verificar si un nodo tiene aristas
salientes tiene_aristas(Nodo) :- arco(Nodo, _, _). % Regla D: Calcular costo para ir de X a
Z pasando por Y costo_via_intermedio(X, Y, Z, Costo) :- arco(X, Y, C1), arco(Y, Z, C2),
Costo is C1 + C2. % Recorrido recursivo en el grafo dirigido camino_dirigido(X, Y, [X,Y],
C) :- arco(X, Y, C). camino_dirigido(X, Y, [X|Resto], Costo) :- arco(X, Z, C1),
camino_dirigido(Z, Y, Resto, C2), Costo is C1 + C2.
```

Respuestas Explicitas a las Preguntas Guía

a) ¿Existe conexión entre Saskatoon y Vancouver?

?- camino_dirigido(saskatoon, vancouver, Ruta, Costo).

Respuesta: false. Al ser un grafo dirigido, no hay arcos que permitan viajar de Saskatoon hacia Vancouver.

b) ¿Con qué nodos está conectado Regina y cuál es el costo?

?- arco(regina, Destino, Costo).

Respuesta: Destino = saskatoon (Costo = 7); Destino = winnipeg (Costo = 4).

e) ¿Es posible viajar desde Edmonton a Calgary? (Pregunta más interesante)

?- camino_dirigido(edmonton, calgary, Ruta, Costo).

Respuesta: true.

Ruta = [edmonton, saskatoon, calgary], Costo Total = 21.

Explicación: No existe un arco directo de Edmonton a Calgary, pero el motor Prolog encuentra mediante backtracking la ruta intermedia pasando por Saskatoon (12 + 9 = 21).

